

ANÁLISIS Y DESARROLLO DE SOFTWARE

INTEGRANTES:

BRANDON ARLEY RESTREPO GÉLVEZ

JOHAN SEBASTIÁN ACOSTA SÁNCHEZ

WILMER JESÚS CONTRERAS RANGEL

FACTUGEST

CÚCUTA, NORTE DE SANTANDER

SENA — CIES

2026

FACTUGEST

Plataforma web de facturación electrónica para la emisión de documentos por cuenta de terceros
mediante una API de integración

Presentado por:

BRANDON ARLEY RESTREPO GÉLVEZ

JOHAN SEBASTIÁN ACOSTA SÁNCHEZ

WILMER JESÚS CONTRERAS RANGEL

Ficha 3115426

Trabajo de grado presentado como requisito para optar al título de:

Tecnólogo en Análisis y Desarrollo de Software

SERVICIO NACIONAL DE APRENDIZAJE — SENA

Centro de la Industria, la Empresa y los Servicios (CIES)

Tecnólogo en Análisis y Desarrollo de Software

CÚCUTA, NORTE DE SANTANDER

2026

Tabla de contenido

RESUMEN	5
ABSTRACT.....	7
INTRODUCCIÓN	9
Capítulo 1. Planteamiento del problema y formulación del proyecto	12
1.1 Descripción del problema	12
1.2 Justificación	13
1.3 Restricciones	15
1.4 Limitaciones.....	15
1.5 Objetivo general.....	16
1.6 Objetivos específicos	16
Capítulo 2. Marco referencial del proyecto	18
2.1 Marco teórico	18
2.2 Marco conceptual.....	22
2.3 Marco tecnológico	24
Capítulo 3. Metodología de investigación y desarrollo	26
3.1 Metodología de desarrollo: Scrum.....	26
3.2 Product backlog	27
3.3 Elicitación de requisitos.....	28
3.4 Técnicas de elicitación utilizadas.....	29

3.4.1 Encuesta.	29
3.4.2 Observación directa.	30
3.7 Equipo de proyecto y partes interesadas	32

RESUMEN

El presente proyecto tiene como propósito el análisis, diseño y desarrollo de FactuGest, una plataforma web de facturación electrónica que opera como proveedor tecnológico para micro, pequeñas y medianas empresas de Colombia. La propuesta surge de una barrera concreta: la facturación electrónica es de obligatorio cumplimiento, pero la mayoría de estas empresas ya opera con un software propio —un punto de venta, un sistema contable o una aplicación hecha a la medida— y adoptar la obligación por la vía habitual implica reemplazarlo, con el costo, la curva de aprendizaje y la interrupción de la operación que ello supone. FactuGest resuelve esa situación emitiendo por cuenta de la empresa a través de una API de integración que su propio sistema consume, de modo que el negocio cumple sin cambiar la herramienta con la que trabaja. La plataforma expide facturas de venta, notas crédito y notas débito, calcula de forma automática bases gravables, descuentos y prorrateo de impuestos, reserva el consecutivo autorizado de la resolución, genera el código único de facturación electrónica, la representación gráfica en PDF y el archivo XML bajo el estándar UBL 2.1, y hace llegar el documento al comprador. Administra además el negocio del proveedor: el registro de las empresas integradas, la llave con la que cada una accede, el plan de suscripción contratado, el control de su consumo mensual contra el cupo de ese plan y la facturación de la mensualidad correspondiente, sobre la cual construye un tablero de control y siete reportes exportables. El proyecto se desarrolló bajo la metodología ágil Scrum, aplicando encuestas y observación directa como técnicas de elicitación de requisitos, a partir de las cuales se definieron ochenta y dos requisitos funcionales, treinta y dos no funcionales y cincuenta y seis casos de uso. La solución se construyó sobre una arquitectura por capas con Python y FastAPI, plantillas Jinja2 y base de datos MySQL, con la lógica de negocio

concentrada en una capa de servicios que alimenta por igual a la aplicación web y a la interfaz de integración.

Palabras clave: facturación electrónica, API de integración, proveedor tecnológico, DIAN, UBL 2.1, middleware, suscripción por volumen.

ABSTRACT

The purpose of this project is the analysis, design, and development of FactuGest, a web-based electronic invoicing platform that operates as a technology provider for micro, small, and medium-sized enterprises in Colombia. The proposal addresses a specific barrier: electronic invoicing is mandatory, yet most of these businesses already run their own software—a point of sale, an accounting system, or a custom application—and complying through the usual route means replacing it, with the associated cost, learning curve, and disruption of daily operations. FactuGest resolves this by issuing documents on behalf of the company through an integration API that the company's own system consumes, so the business complies without changing the tool it works with. The platform issues sales invoices, credit notes, and debit notes; automatically calculates taxable bases, discounts, and tax proration; reserves the authorized sequential number from the tax resolution; generates the unique electronic invoicing code, the PDF representation, and the XML file under the UBL 2.1 standard; and delivers the document to the buyer. It also manages the provider's own business: registration of integrated companies, the key each one uses to authenticate, the subscription plan contracted, the monitoring of monthly consumption against that plan's quota, and the billing of the corresponding monthly fee, upon which it builds a control dashboard and seven exportable reports. The project was developed using the Scrum agile methodology, applying surveys and direct observation as requirements elicitation techniques, from which eighty-two functional requirements, thirty-two non-functional requirements, and fifty-six use cases were defined. The solution was built on a layered architecture using Python and FastAPI, Jinja2 templates, and a MySQL database, with business logic concentrated in a service layer that feeds both the web application and the integration interface.

Keywords: electronic invoicing, integration API, technology provider, DIAN, UBL 2.1, middleware, volume-based subscription.

INTRODUCCIÓN

La facturación electrónica dejó de ser en Colombia una opción de modernización para convertirse en una obligación tributaria exigible. Su incumplimiento no es una omisión menor: compromete la deducibilidad de costos y gastos, dificulta las relaciones comerciales con clientes que exigen un soporte válido y expone al contribuyente a sanciones. Sin embargo, la carga que impone no se distribuye por igual. Una empresa grande incorpora la obligación en un sistema que ya administra sus procesos; un negocio pequeño se encuentra con una exigencia técnica que no sabe cómo atender y con un costo que no tenía previsto.

El diagnóstico que dio origen a este proyecto encontró que la dificultad principal no era la falta de una herramienta de facturación, sino algo distinto: la mayoría de estos negocios ya opera con un software propio. Un punto de venta en la tienda, un sistema contable en la oficina, una aplicación hecha a la medida años atrás. Adoptar la facturación electrónica por la vía habitual —un sistema nuevo que la incluya— significa reemplazar esa herramienta, capacitar de nuevo al personal e interrumpir la operación mientras tanto. La barrera no es tecnológica sino de reemplazo, y por eso una solución que exija cambiar de software no la resuelve: la traslada.

En respuesta a esa situación surge FactuGest, una plataforma web que opera como proveedor tecnológico de facturación electrónica. En lugar de sustituir el sistema de la empresa, se conecta con él: expone una interfaz de integración que el software existente consume para enviar los datos de una venta, y FactuGest se encarga de expedir el documento electrónico con su numeración autorizada, su código único, su representación gráfica y su archivo XML, y de hacerlo llegar al comprador. La empresa cumple con la obligación conservando la herramienta con la que trabaja.

Prestar ese servicio implica administrarlo. La plataforma incorpora, junto al motor de emisión, los módulos que sostienen el negocio del proveedor: el registro de las empresas integradas y de la llave con que cada una se identifica, los planes de suscripción por volumen de documentos, el control del consumo mensual contra el cupo contratado y la facturación de la mensualidad correspondiente. Sobre esa operación se construyeron un tablero de control y un conjunto de reportes exportables que permiten evaluar tanto la prestación del servicio como su recaudo.

El desarrollo se fundamentó en conceptos relacionados con la facturación electrónica y su normativa, los estándares de documentos comerciales en formato XML, las arquitecturas orientadas a servicios y el modelo de integración por middleware, la seguridad en la autenticación de sistemas, el diseño de bases de datos relacionales y los modelos de negocio basados en suscripción. Estos elementos proporcionaron el marco conceptual y tecnológico necesario para diseñar una solución que respondiera al problema identificado.

Para asegurar que la solución atendiera necesidades reales y no supuestas, se aplicaron técnicas de elicitación de requisitos basadas en encuestas dirigidas a empresas obligadas a facturar electrónicamente y en observación directa de su operación. La información recopilada permitió establecer ochenta y dos requisitos funcionales y treinta y dos no funcionales, a partir de los cuales se definieron los actores, los casos de uso, las historias de usuario y los módulos que orientaron la construcción del sistema.

La metodología Scrum se empleó como marco de trabajo para organizar el proyecto, gestionar el product backlog, planificar las iteraciones y hacer seguimiento al avance de cada módulo. Su carácter iterativo resultó determinante en un aspecto concreto: el alcance del proyecto cambió durante el desarrollo. Lo que empezó como un sistema de facturación para una

empresa se convirtió en un proveedor que emite por cuenta de varias, al comprobarse que el reemplazo del software era la verdadera barrera. Un marco de trabajo capaz de incorporar ese hallazgo sin desechar lo construido fue lo que permitió que el cambio de rumbo fuera una corrección y no un reinicio.

El presente documento describe las etapas desarrolladas durante la construcción de FactuGest: la identificación de la problemática y la definición de los objetivos, el marco referencial que sustenta la solución, la metodología de desarrollo y las técnicas de elicitación aplicadas, el análisis y la especificación de los requisitos, el diseño de la arquitectura y de la base de datos, la implementación de los módulos y, finalmente, las pruebas realizadas y los resultados obtenidos.

Capítulo 1. Planteamiento del problema y formulación del proyecto

1.1 Descripción del problema

La facturación electrónica es en Colombia un requisito de obligatorio cumplimiento para los sujetos definidos por la Dirección de Impuestos y Aduanas Nacionales. El incumplimiento compromete la deducibilidad de costos y gastos, dificulta las relaciones comerciales con clientes que exigen un soporte válido y expone al contribuyente a sanciones. La obligación, sin embargo, no se distribuye por igual: una empresa grande la incorpora en el sistema que ya administra sus procesos, mientras que un negocio pequeño se enfrenta a una exigencia técnica que no sabe cómo atender y a un costo que no tenía previsto.

El diagnóstico realizado sobre micro, pequeñas y medianas empresas de la región identificó que la dificultad principal no es la ausencia de una herramienta de facturación, sino la condición en que esa herramienta llega. La mayoría de estos negocios ya opera con un software propio: un punto de venta en el mostrador, un sistema contable en la oficina o una aplicación desarrollada a la medida años atrás. Ese software concentra su catálogo, sus clientes, sus precios y la manera en que el personal aprendió a trabajar.

Frente a esa situación, la oferta habitual del mercado consiste en un sistema nuevo que incluye la facturación electrónica. Adoptarlo implica reemplazar la herramienta existente, migrar la información, capacitar de nuevo al personal e interrumpir la operación mientras dura el cambio. El problema, entonces, no es tecnológico sino de reemplazo: la empresa no rechaza facturar electrónicamente, rechaza abandonar el sistema con el que ya trabaja. Una solución que exija ese cambio no resuelve la barrera, la traslada.

A esta dificultad se suman otras tres, observadas en los negocios que sí intentaron cumplir. La primera es el desconocimiento de las condiciones técnicas que la normativa impone

—numeración autorizada por resolución, código único por documento, archivo XML bajo un estándar determinado y entrega al comprador—, que no forman parte del oficio de quien atiende un negocio. La segunda es el costo de las soluciones disponibles, dimensionado para empresas con volúmenes de facturación muy superiores. La tercera es que la información que la empresa está obligada a registrar termina dispersa entre el software propio y el del proveedor de facturación, sin que ninguno de los dos ofrezca una vista completa de la operación.

De lo anterior surge la pregunta que orienta este proyecto: ¿cómo puede una micro, pequeña o mediana empresa cumplir con la obligación de facturar electrónicamente ante la DIAN sin reemplazar el software con el que ya opera y sin asumir el costo técnico y económico que las soluciones disponibles le imponen?

1.2 Justificación

Se propone el desarrollo de una plataforma que opere como proveedor tecnológico de facturación electrónica y que se integre con el software que la empresa ya utiliza, en lugar de sustituirlo. La decisión de operar bajo un modelo de middleware —una interfaz que se conecta con los sistemas existentes— responde directamente a la barrera identificada en el diagnóstico: preserva la inversión previa del empresario, elimina la curva de aprendizaje de un sistema nuevo y evita interrumpir la operación diaria del negocio.

Desde el punto de vista normativo, la solución atiende una necesidad jurídicamente exigible y con consecuencias económicas directas. Facilitar el cumplimiento no es una comodidad: es la diferencia entre un negocio que puede deducir sus costos y sostener relaciones comerciales formales y otro que queda expuesto a sanciones y excluido de clientes que exigen soporte válido.

En el plano social y económico, las micro, pequeñas y medianas empresas constituyen la base del aparato productivo colombiano y son, a la vez, las que menos recursos tienen para asumir la carga técnica de la transformación digital tributaria. Una solución accesible, cobrada por volumen de documentos y no por licencia, permite que un negocio pequeño opere en la formalidad en las mismas condiciones técnicas que una empresa grande. El efecto no se limita al empresario: una mayor formalización fortalece el recaudo y mejora la trazabilidad de las operaciones comerciales.

En lo técnico, el modelo elegido reduce el esfuerzo de integración a su mínimo. El sistema de la empresa envía los datos de una venta y recibe el documento ya emitido, sin ocuparse de la numeración autorizada, del cálculo tributario, del estándar del archivo XML ni de la entrega al comprador. Que el contrato de la interfaz se publique de forma automática permite que quien integra construya su cliente sin depender de documentación escrita aparte, que es donde suelen aparecer las discrepancias entre lo documentado y lo que el sistema realmente hace.

Desde la gestión empresarial, la solución convierte una obligación tributaria en un activo de información. Los datos que la empresa está legalmente obligada a registrar se transforman en indicadores de operación y de cartera sin trabajo adicional para el usuario, lo que agrega valor más allá del cumplimiento y responde a una necesidad real de control en negocios que hoy no cuentan con herramientas de análisis.

Finalmente, en el ámbito formativo, el proyecto integra y evidencia las competencias del programa: levantamiento y especificación de requisitos, modelado de casos de uso, diseño y normalización de bases de datos relacionales, desarrollo de una aplicación web, diseño e implementación de servicios REST, aplicación de mecanismos de seguridad, control de

versiones y trabajo bajo una metodología ágil. Su desarrollo sobre un problema real, con normativa vigente y usuarios reales, otorga al ejercicio un nivel de exigencia equivalente al de un entorno productivo.

1.3 Restricciones

Las restricciones son condiciones impuestas al proyecto desde fuera, que delimitan lo que puede construirse y bajo qué condiciones.

- El tiempo disponible para el desarrollo está delimitado por el cronograma del programa de formación.
- La emisión en los servicios de producción de la administración tributaria depende de la habilitación del proveedor, trámite que no está bajo control del equipo. Mientras tanto, el código único de facturación electrónica se genera en modalidad de pruebas.
- La numeración de cada documento depende de una resolución de facturación vigente, con prefijo y rango autorizados a la empresa emisora. El sistema no puede suplirla.
- El proyecto no puede modificar el software de la empresa integrada: la solución debe limitarse a ofrecer una interfaz que ese software consuma.
- El funcionamiento requiere conexión a internet en los dos extremos, el del proveedor y el del sistema integrado.
- La infraestructura disponible durante el desarrollo corresponde a un entorno local, no a un servidor de producción con sus condiciones de disponibilidad y respaldo.

1.4 Limitaciones

Las limitaciones son alcances que la solución no cubre o factores que restringen sus resultados. Se declaran de forma explícita porque un sistema que expide documentos con efectos tributarios no admite ambigüedad sobre lo que hace y lo que no.

- La adopción de la solución exige que la empresa cuente con un sistema capaz de realizar peticiones HTTP y con alguien que lo ajuste. Un negocio cuyo software no admita modificaciones no puede integrarse por esta vía.
- La entrega del documento al comprador depende de un servicio de correo de terceros, cuya indisponibilidad el sistema no controla.
- El recaudo de las mensualidades se registra manualmente: no se implementó integración con pasarelas de pago.
- El rendimiento del sistema se verificó en condiciones de desarrollo. No fue sometido a una carga de producción sostenida ni a un número elevado de emisiones simultáneas.
- El cliente móvil quedó fuera del alcance de esta versión, al concentrarse el esfuerzo en la interfaz de integración.
- Delegar la facturación en un tercero supone una decisión de confianza que excede lo técnico. Una empresa puede rechazar la solución por esa razón aunque funcione correctamente.

1.5 Objetivo general

Desarrollar FactuGest, una plataforma web de facturación electrónica que opere como proveedor tecnológico para micro, pequeñas y medianas empresas de Colombia, permitiéndoles emitir facturas de venta, notas crédito y notas débito conforme a la normativa de la DIAN a través de una API de integración que se conecta con los sistemas que ya utilizan.

1.6 Objetivos específicos

1. Implementar el módulo de emisión de documentos electrónicos que genere facturas de venta, notas crédito y notas débito con el cálculo automático de bases gravables,

descuentos y prorrateo de IVA, asignando a cada documento su consecutivo autorizado, su CUFE, su representación gráfica en PDF y su archivo XML bajo el estándar UBL 2.1.

2. Construir una API REST de integración que exponga los servicios de facturación a sistemas externos, con autenticación por llave individual, control de cupo por plan y una forma única de respuesta y de error documentada en OpenAPI.
3. Desarrollar el módulo de clientes API y planes de suscripción que administre el alta de empresas integradas, la generación y rotación de sus llaves, el control de su consumo mensual y la facturación de su mensualidad.
4. Implementar un panel de control y un módulo de reportes exportables a CSV y PDF que consoliden la operación del servicio y la situación financiera de la empresa.
5. Establecer el esquema de seguridad de la plataforma mediante autenticación de usuarios, roles jerárquicos con control de acceso por ruta y un registro de auditoría de las operaciones de escritura.

Capítulo 2. Marco referencial del proyecto

2.1 Marco teórico

El marco teórico reúne los fundamentos sobre los que se sostiene la solución propuesta: la naturaleza de la facturación electrónica como sistema de información con efectos jurídicos, el modelo de intermediación que permite integrarla sin reemplazar el software existente, los estilos de arquitectura empleados y los principios de seguridad y de calidad que rigen su construcción.

La facturación electrónica como sistema de información tributario. La factura electrónica no es la versión digital de un documento impreso. Es un documento con validez jurídica cuya estructura, numeración y contenido están determinados por la norma (Estatuto Tributario, art. 616-1). La reglamentación colombiana establece las condiciones de generación, transmisión y entrega de estos documentos, así como la figura del proveedor tecnológico, habilitado para prestar esos servicios a otras empresas (DIAN, 2020). De esta naturaleza normativa se desprende una consecuencia técnica que atraviesa todo el proyecto: los errores no degradan el servicio, lo invalidan. Un consecutivo repetido o un archivo que no cumple el estándar no producen un documento defectuoso, producen un documento rechazado.

Estándares de documentos comerciales electrónicos. Para que un documento pueda ser procesado automáticamente por sistemas distintos, su contenido debe expresarse en una estructura común. El estándar Universal Business Language define, en formato XML, la representación de los documentos comerciales —entre ellos la factura, la nota crédito y la nota débito— y es el adoptado por la normativa colombiana (OASIS, 2013). Trabajar sobre un estándar y no sobre un formato propio es lo que permite que un documento emitido por una plataforma sea legible por la autoridad tributaria y por el sistema del comprador sin acuerdos previos entre ellos.

Arquitectura orientada a servicios y el modelo de middleware. Un middleware es un componente que se ubica entre dos sistemas y les permite comunicarse sin que ninguno modifique su funcionamiento interno. Aplicado a este proyecto, significa que la empresa conserva el software con el que opera y que la plataforma se encarga únicamente de convertir los datos de una venta en un documento electrónico válido. La integración se resuelve mediante servicios expuestos por la red, un enfoque que permite que sistemas construidos con tecnologías distintas colaboren a través de contratos explícitos (Sommerville, 2011).

El estilo arquitectónico REST. REST es un estilo de arquitectura para sistemas distribuidos que organiza la interacción alrededor de recursos identificados por una dirección, sobre los que se actúa con un conjunto reducido de operaciones y en el que cada petición contiene por sí sola la información necesaria para ser atendida (Fielding, 2000). Esa última propiedad —que el servidor no conserve estado de conversación entre peticiones— es la que hace que una interfaz de este tipo pueda ser consumida por un punto de venta, por un sistema contable o por una aplicación móvil sin distinción (Richardson y Ruby, 2007). El contrato de la interfaz puede además describirse en un formato legible por máquinas, lo que permite generar documentación y clientes de forma automática a partir de la misma definición (OpenAPI Initiative, 2021).

Arquitectura por capas y separación de responsabilidades. La organización del software en capas con responsabilidades separadas —presentación, lógica de negocio y acceso a datos— permite que un cambio en una de ellas no obligue a modificar las demás (Pressman y Maxim, 2020). En este proyecto la separación cumple además una función concreta: la misma lógica de negocio alimenta la aplicación web y la interfaz de integración, de modo que una regla

tributaria se implementa una sola vez y rige por igual para las dos. Duplicarla habría producido, tarde o temprano, dos resultados distintos para la misma operación.

El modelo relacional de datos. El modelo relacional organiza la información en relaciones compuestas por atributos, e independiza la forma en que los datos se consultan de la forma en que se almacenan (Codd, 1970). Sus mecanismos de integridad —claves primarias, claves foráneas y restricciones de unicidad— no son una formalidad de diseño en un sistema como este: son lo que impide que dos documentos del mismo emisor compartan número o que una línea quede huérfana de su documento.

Seguridad: autenticación y control de acceso. La protección de un sistema de información se apoya en dos mecanismos distintos. La autenticación establece quién realiza una petición; el control de acceso determina qué puede hacer una vez identificado. El control de acceso basado en roles asigna los permisos a roles y no a personas, de modo que administrar el acceso de un usuario se reduce a asignarle el rol que corresponde (Sandhu et al., 1996). Respecto de las contraseñas, la práctica aceptada es no almacenarlas: se guarda el resultado de una función de derivación de costo ajustable, que puede endurecerse con el tiempo sin cambiar el esquema (Provos y Mazières, 1999).

Calidad del software. La calidad de un producto de software puede describirse mediante características medibles —adecuación funcional, eficiencia, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad— que sirven tanto para especificar como para evaluar (ISO/IEC, 2011). Los requisitos no funcionales de este proyecto se organizaron siguiendo esa lógica, y cada uno se acompaña de la forma en que se comprueba: un atributo de calidad que no indica cómo verificarse no puede declararse cumplido.

Modelos de negocio basados en suscripción. En el modelo de software como servicio, el cliente no adquiere una licencia sino el derecho a usar un servicio durante un periodo, y el proveedor conserva la responsabilidad de operarlo. Cuando la tarifa se asocia al volumen consumido, el costo de entrada disminuye y se vuelve proporcional al tamaño del negocio, lo que resulta determinante para una empresa pequeña: paga por los documentos que emite y no por una licencia dimensionada para volúmenes que no maneja.

2.2 Marco conceptual

Se relacionan a continuación los términos empleados a lo largo del documento, con el significado preciso que tienen dentro del dominio de la facturación electrónica y de este proyecto en particular.

- **Factura electrónica de venta:** documento que soporta una operación de venta de bienes o servicios, generado y transmitido en formato electrónico conforme a las condiciones que fija la administración tributaria.
- **Nota crédito:** documento electrónico que corrige una factura ya expedida disminuyendo su valor, por devolución, anulación o descuento posterior.
- **Nota débito:** documento electrónico que corrige una factura ya expedida aumentando su valor, por intereses, gastos o mayores valores no incluidos.
- **CUFE:** código único de facturación electrónica. Valor irrepetible que identifica cada documento y permite verificar que su contenido no fue alterado.
- **Resolución de facturación:** acto administrativo mediante el cual la autoridad tributaria autoriza a una empresa un prefijo y un rango de numeración, por un periodo determinado.
- **Consecutivo:** número que ocupa un documento dentro del rango autorizado. No puede repetirse ni saltarse arbitrariamente.
- **Emisor:** empresa por cuenta de la cual se expide el documento y cuya identificación aparece en él.
- **Adquiriente o comprador:** persona natural o jurídica que recibe el bien o el servicio y a quien va dirigido el documento.
- **Proveedor tecnológico:** persona jurídica que presta a otras empresas los servicios de generación, transmisión y entrega de documentos electrónicos.

- **Middleware:** componente que se sitúa entre dos sistemas y les permite comunicarse sin que ninguno de los dos tenga que modificar su funcionamiento interno.
- **API:** interfaz de programación de aplicaciones. Conjunto de operaciones que un sistema expone para que otro sistema las consuma.
- **Llave de API:** credencial con la que un sistema externo se identifica ante otro. Sustituye al usuario y la contraseña cuando quien se autentica es un programa y no una persona.
- **Idempotencia:** propiedad por la cual repetir una operación produce el mismo resultado que ejecutarla una sola vez. Es lo que permite reintentar una petición sin duplicar el documento.
- **Plan de suscripción:** modalidad de contratación en la que el cliente paga una tarifa periódica que incluye una cantidad determinada de documentos.
- **Cupo:** cantidad de documentos que el plan contratado incluye dentro del mes.
- **Excedente:** documento emitido por encima del cupo del plan, que se cobra de forma independiente.
- **Auditoría:** registro de las operaciones realizadas sobre el sistema, con el usuario que las hizo y el momento en que ocurrieron.
- **Trazabilidad:** posibilidad de reconstruir la historia de un dato o de un documento a partir de los registros que el sistema conserva.
- **Rol:** conjunto de permisos que determina a qué funciones y a qué información puede acceder un usuario.
- **Tablero de control:** pantalla que reúne los indicadores de la operación y los presenta de forma gráfica para apoyar la toma de decisiones.

2.3 Marco tecnológico

Se describen las tecnologías empleadas en la construcción de la solución, indicando la función que cumple cada una dentro del sistema.

- **Arquitectura cliente-servidor:** modelo en el que un servidor centraliza el procesamiento y el almacenamiento, y los clientes —navegadores o sistemas externos— consumen sus servicios por la red.
- **Python:** lenguaje de programación empleado para construir la totalidad del lado del servidor. Se eligió por su legibilidad y por la madurez de sus bibliotecas para generación de documentos y acceso a datos.
- **FastAPI:** framework web sobre el que se construyeron tanto las rutas de la aplicación como la API de integración. Valida los datos de entrada a partir de tipos declarados y publica de forma automática el contrato de la interfaz, lo que reduce el esfuerzo de quien integra.
- **Uvicorn:** servidor de aplicación que ejecuta el proyecto y atiende las peticiones HTTP.
- **Pydantic:** biblioteca de validación con la que se declaran los modelos de entrada y salida de la API. Un dato que no cumple el modelo no llega a la lógica de negocio.
- **Jinja2:** motor de plantillas con el que se generan las páginas de la aplicación web a partir de los datos que entrega el servidor.
- **Bootstrap:** biblioteca de estilos utilizada para la interfaz del panel, que le da un comportamiento adaptable a distintos tamaños de pantalla.
- **Chart.js:** biblioteca de gráficas empleada en el tablero de control para presentar las series de documentos emitidos y de facturación.

- **MySQL:** sistema gestor de base de datos relacional donde reside toda la información del sistema. Se accede mediante consultas SQL explícitas, sin mapeador objeto-relacional, para conservar el control sobre cada consulta.
- **ReportLab:** biblioteca con la que se construye la representación gráfica en PDF de cada documento emitido.
- **bcrypt:** algoritmo de derivación con el que se calcula el hash de las contraseñas. Su costo es ajustable, de modo que puede endurecerse a medida que el hardware avanza.
- **OpenAPI y Swagger:** especificación e interfaz con las que el contrato de la API queda publicado y explorable, generadas a partir del propio código.
- **Git y GitHub:** sistema de control de versiones y plataforma de alojamiento remoto empleados para gestionar los cambios del código y el trabajo colaborativo.
- **PyCharm:** entorno de desarrollo utilizado para la escritura, depuración y mantenimiento del código fuente.
- **pytest:** herramienta con la que se ejecutan las pruebas automatizadas sobre la aritmética tributaria, el contrato de la API y la generación de documentos.

Capítulo 3. Metodología de investigación y desarrollo

3.1 Metodología de desarrollo: Scrum

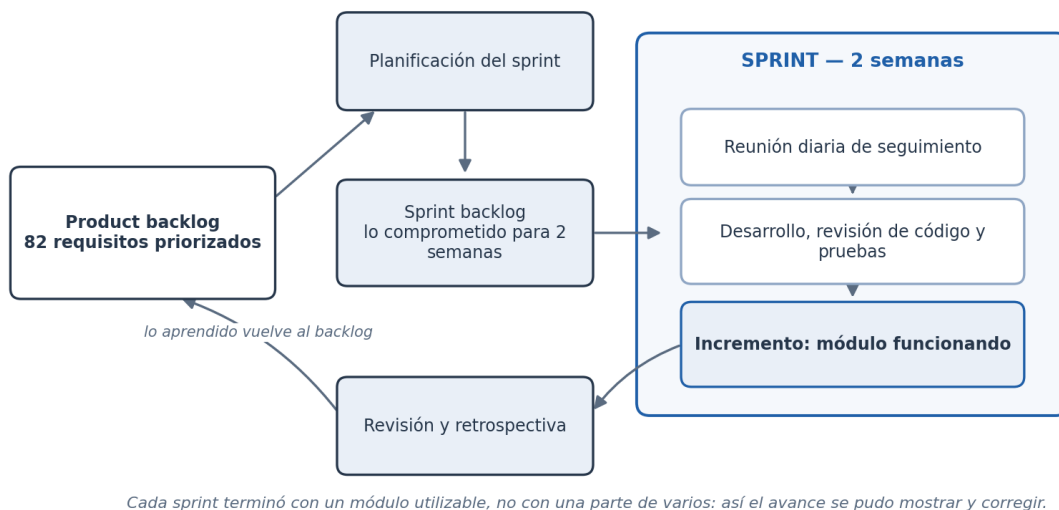
Scrum es un marco de trabajo ágil para el desarrollo de productos complejos, basado en iteraciones cortas de duración fija llamadas sprints, al término de cada una de las cuales se entrega un incremento utilizable del producto (Schwaber y Sutherland, 2020). Su estructura se apoya en tres artefactos —el product backlog, el sprint backlog y el incremento— y en eventos de planificación, seguimiento diario, revisión y retrospectiva.

La elección de Scrum para este proyecto no respondió únicamente a su difusión. Respondió a una condición del problema: al inicio del desarrollo no estaba resuelto de qué manera la solución llegaría a las empresas. Un marco de trabajo que exigiera fijar la totalidad de los requisitos antes de construir habría obligado a comprometerse con esa decisión demasiado pronto.

El hecho más importante del proyecto confirma esa elección. Durante el desarrollo se comprobó que la barrera real no era la ausencia de una herramienta de facturación sino el reemplazo del software existente, y el proyecto pasó de ser un sistema de facturación para una empresa a ser un proveedor que emite por cuenta de varias a través de una API. Ese cambio se incorporó como una reordenación del product backlog: lo construido —el motor de emisión, el cálculo tributario, la generación del PDF y del XML— siguió siendo válido, porque el cambio afectó la manera de ofrecer el servicio y no la manera de expedir un documento. Con un enfoque secuencial el mismo hallazgo, aparecido a mitad de la construcción, habría implicado rehacer el análisis completo.

Figura 1

Ciclo de Scrum aplicado al desarrollo de FactuGest



Nota. Elaboración propia con base en Schwaber y Sutherland (2020). Los sprints tuvieron una duración de dos semanas y cada uno cerró con un módulo utilizable.

Se definieron ocho sprints de dos semanas durante la fase de ejecución. Cada uno se orientó a dejar un módulo en funcionamiento y no una porción de varios: un módulo terminado puede mostrarse, probarse y corregirse, mientras que tres módulos a medias solo pueden describirse.

3.2 Product backlog

El product backlog se conformó con las funcionalidades identificadas durante la elicitación, organizadas en los ocho módulos funcionales del sistema y priorizadas según su valor para el negocio y su urgencia, criterio que se detalla en el capítulo siguiente. La lista completa comprende ochenta y dos requisitos funcionales.

El orden de los sprints no siguió la prioridad de los requisitos sino sus dependencias técnicas, que es una distinción que conviene explicar. El módulo de emisión es el de mayor prioridad del proyecto, pero no puede construirse antes que el catálogo de empresas emisoras: sin

una resolución con su prefijo y su rango no hay número que reservar. De la misma manera, la facturación de las mensualidades se dejó para el sprint 7 porque se emite a través de la propia API, que solo existió desde el sprint 6.

Tabla 1

Organización del product backlog en sprints

Sprint	Módulo	Alcance comprometido	Requisitos
Sprint 1	Seguridad y acceso	Autenticación de usuarios, hash de contraseñas, roles jerárquicos, control de acceso por ruta y cierre de sesión por inactividad.	RF 7
Sprint 2	Configuración y catálogos	Empresas emisoras con su resolución y su rango, impuestos, descuentos, métodos y estados de pago, y catálogo de ubicación.	RF 6
Sprint 3	Motor de emisión	Cálculo tributario, reserva atómica del consecutivo, generación del CUFE, del PDF con el membrete del emisor y del XML bajo UBL 2.1.	RF 2
Sprint 4	Facturación y cartera propias	Catálogo de planes y servicios, clientes, emisión de la factura propia, registro de pagos y estados de cartera.	RF 4
Sprint 5	Clientes API y llaves	Alta de empresas integradas, generación y rotación de llaves, estados del cliente y registro en auditoría.	RF 1
Sprint 6	API de integración	Autenticación por llave, validación de la entrada, forma única de error, operaciones de emisión y consulta, y publicación del contrato en OpenAPI.	RF 8
Sprint 7	Consumo, cupo y planes	Conteo del consumo mensual, bloqueo por cupo agotado, periodo facturable, excedentes y emisión de la mensualidad por la propia API.	RF 3
Sprint 8	Tablero, reportes y auditoría	Indicadores del servicio y de la venta, filtros de periodo y empresa, los siete reportes, su exportación y la consulta del rastro de auditoría.	RF 5

Nota. Elaboración propia. La columna «Requisitos» remite a los módulos del catálogo de requisitos funcionales especificado en el capítulo 4.

3.3 Elicitación de requisitos

La elicitación de requisitos es la etapa en la que se identifican, comprenden y documentan las necesidades de los usuarios y las expectativas de los interesados. De su calidad depende que el sistema resuelva un problema real y no uno supuesto: un error introducido aquí se propaga al diseño, a la construcción y a las pruebas, y resulta más costoso de corregir en cada etapa que atraviesa (Sommerville, 2011).

En este proyecto la elicitación debía responder una pregunta previa a cualquier listado de funcionalidades: por qué empresas obligadas a facturar electrónicamente no lo estaban haciendo. Preguntar directamente qué funciones querían en un sistema de facturación habría producido una lista de funcionalidades convencional y habría dejado intacta la causa del incumplimiento.

Se emplearon dos técnicas complementarias: la encuesta, para conocer la situación de un conjunto amplio de negocios, y la observación directa, para ver de cerca cómo opera uno de ellos. La primera indica qué tan extendido está un problema; la segunda, por qué ocurre.

3.4 Técnicas de elicitación utilizadas

3.4.1 Encuesta.

Se diseñó un cuestionario dirigido a micro, pequeñas y medianas empresas de la ciudad, con el propósito de establecer su situación frente a la obligación de facturar electrónicamente, el software con el que operan actualmente y las razones por las cuales no han adoptado la facturación electrónica o han tenido dificultades para hacerlo.

El instrumento consta de doce preguntas de respuesta cerrada, agrupadas en cuatro bloques: situación frente a la obligación, herramientas que utiliza el negocio, barreras percibidas y disposición frente a una solución que se integre con el software existente. Las preguntas 6 y 7 son deliberadamente complementarias: la primera indaga si la empresa cambiaría de software para poder cumplir y la segunda si le interesaría cumplir sin cambiarlo. Contrastar ambas respuestas es lo que permite distinguir el rechazo a la facturación electrónica del rechazo al reemplazo del software, que son cosas distintas y conducen a soluciones distintas.

Tabla 2

Instrumento de encuesta aplicado

N.º	Pregunta	Opciones de respuesta
1	¿Su empresa está obligada a facturar electrónicamente ante la DIAN?	Sí / No / No lo sé

N.º	Pregunta	Opciones de respuesta
2	¿Actualmente emite facturación electrónica?	Sí / No / Está en proceso
3	¿Con qué software administra hoy sus ventas?	Punto de venta / Sistema contable / Aplicación propia / Hojas de cálculo / Ninguno
4	¿Cuántos documentos emite en promedio al mes?	Menos de 50 / 50 a 150 / 151 a 400 / Más de 400
5	¿Cuál ha sido la principal dificultad para adoptar la facturación electrónica?	El costo / Tener que cambiar de software / No saber cómo hacerlo / La capacitación del personal / Ninguna
6	¿Estaría dispuesto a cambiar el software con el que trabaja hoy para poder facturar electrónicamente?	Sí / No / Tal vez
7	Si pudiera seguir usando su software actual y que este emitiera la factura electrónica automáticamente, ¿le interesaría?	Sí / No / Tal vez
8	¿Quién se encarga en su empresa de los temas de sistemas?	Un empleado propio / Un técnico externo / El proveedor del software / Nadie
9	¿Qué tan importante considera recibir un soporte oportuno ante un fallo en la facturación?	Escala de 1 a 5
10	¿Cuánto estaría dispuesto a pagar mensualmente por el servicio de facturación electrónica?	Menos de \$50.000 / \$50.000 a \$100.000 / \$100.001 a \$200.000 / Más de \$200.000
11	¿Le interesaría contar con reportes de sus ventas a partir de lo que factura?	Sí / No / Tal vez
12	¿Ha tenido que corregir o anular facturas ya emitidas?	Nunca / Rara vez / Con frecuencia

Nota. Elaboración propia. Cuestionario de respuesta cerrada, diligenciado en formulario digital.

3.4.2 Observación directa.

La observación directa se realizó sobre la operación de Siste Soluciones, una empresa que administra sus ventas con un punto de venta propio y que no emite facturación electrónica. La técnica consistió en presenciar el registro de ventas durante la jornada, sin intervenir en el proceso, y en revisar con el responsable el recorrido completo de una venta desde que se registra hasta que se entrega el comprobante.

La observación permitió establecer cuatro hechos que una encuesta no habría revelado. El primero es que el punto de venta concentra información que el negocio no está dispuesto a migrar: catálogo, precios, clientes frecuentes y una forma de operar que el personal domina. El segundo es que el registro de una venta toma pocos segundos, de modo que cualquier solución que agregue pasos al proceso de venta será rechazada por el operario aunque la gerencia la apruebe. El tercero es que el comprobante que hoy se entrega no tiene validez fiscal, pero cumple

una función comercial que el cliente ya espera. El cuarto es que el negocio no cuenta con personal técnico permanente: cualquier integración debe ser lo bastante simple como para que la realice quien mantiene el punto de venta.

De estos hallazgos se derivaron decisiones concretas del diseño. Que la emisión ocurra a través de una interfaz que el punto de venta consume —y no de una pantalla adicional que el cajero deba diligenciar— responde al segundo hallazgo. Que el contrato de esa interfaz se publique de forma automática y explorable responde al cuarto. Y que la solución no exija migrar el catálogo ni los clientes del negocio responde al primero.

3.7 Equipo de proyecto y partes interesadas

El equipo de trabajo se conformó con tres integrantes, quienes asumieron los roles previstos por Scrum sin dejar de participar en el desarrollo. En un equipo de este tamaño la separación estricta de roles resulta impracticable: lo que se mantuvo fue la responsabilidad sobre cada función, no la exclusividad en ella.

Tabla 3

Equipo de proyecto y responsabilidades

Integrante	Rol	Responsabilidades
Brandon Arley Restrepo Gélvez	Product Owner y desarrollador	Definió y priorizó el product backlog, mantuvo la relación con la empresa colaboradora y desarrolló el motor de emisión y la API de integración.
Johan Sebastián Acosta Sánchez	Scrum Master y desarrollador	Facilitó las reuniones del equipo, hizo seguimiento a los impedimentos y desarrolló los módulos de facturación propia, consumo y planes.
Wilmer Jesús Contreras Rangel	Analista y desarrollador	Condujo la elicitación de requisitos y el modelado, y desarrolló los módulos de seguridad, configuración, tablero y reportes.

Nota. Elaboración propia.

Las partes interesadas comprenden a quienes se ven afectados por el proyecto o influyen en él, participen o no en su construcción. Su identificación temprana permite anticipar qué se espera de la solución desde cada posición, que no siempre coincide.

Tabla 4

Partes interesadas del proyecto

Parte interesada	Relación con el proyecto
Empresas obligadas a facturar electrónicamente	Usuarios finales del servicio. De su capacidad para integrarse depende que la solución resuelva el problema planteado.
Siste Soluciones	Empresa colaboradora con un punto de venta en operación sin facturación electrónica. Sirvió como caso de integración para verificar que un sistema externo puede emitir a través de la plataforma.
Administración tributaria (DIAN)	Fija las condiciones que el documento debe cumplir y habilita a los proveedores tecnológicos. No participa en el desarrollo, pero determina sus reglas.
Compradores	Reciben el documento emitido. No operan el sistema, pero son quienes verifican que el documento llegue y sea legible.
Equipo de desarrollo	Responsable del análisis, la construcción y las pruebas de la solución.

Parte interesada	Relación con el proyecto
Centro de formación e instructores	Orientan el desarrollo del proyecto y evalúan su cumplimiento como trabajo de grado.

Nota. Elaboración propia. La matriz de stakeholders del capítulo 4 detalla su nivel de interés y de influencia.